



← Back to student guides

 GitHub Actions

CI/CD

3 levels

139 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

EVENT

WORKFLOW

JOB

STEP

Part three of three. The problems at this level are rarely about syntax. They are about a pipeline sixty people depend on, a credential nobody remembers creating, and a failure that started at 09:00 everywhere at once. Start with the error table, then the practices and playbooks underneath it.





These cause incidents rather than red builds.

Symptom	Real cause	Fix
A fork PR could read your secrets	<code>pull_request_target</code> checking out <code>head.sha</code>	Never check out the head ref there; split across <code>pull_request</code> + <code>workflow_run</code>
A PR title executed a command	<code>\${{ github.event.* }}</code> interpolated into a <code>run</code> body	Pass event data through <code>env</code> and quote the variable
A <code>github-script</code> block ran attacker input	The payload was spliced into JavaScript source	Read from <code>process.env</code> at runtime
A compromised step pushed to <code>main</code>	No <code>permissions</code> block, so the token was write-capable	<code>contents: read</code> floor, widened per job
Any branch of any repo can assume the deploy role	OIDC <code>sub</code> claim wildcarded to <code>repo:org/*:*</code>	Constrain to a ref, or better an environment
An action changed under you overnight	Referenced by a mutable tag	SHA-pin, plus Dependabot so pins stay fresh
Pinned actions are years out of date	Pinned but never updated	Dependabot with grouped updates
A cache poisoned a trusted run	A PR branch wrote a cache <code>main</code> later restored	Never restore-and-execute unverified
Automation acts as a departed employee	A PAT instead of a scoped App token	GitHub App installation token
Cross-repo automation fails	<code>GITHUB_TOKEN</code> cannot reach another repository	App token, or a fine-grained PAT
Nobody can say what version is deployed	Deployed a moving tag	Immutable <code>:sha-...</code> tag or a digest



Production runs code CI never tested	image rebuilt per environment	build once, promote the same digest
Docker builds are slow despite a cache	<code>mode=min</code> , so intermediate layers are not exported	<code>cache-to: type=gha,mode=max</code>
A self-hosted runner was compromised	Fork PR scheduled onto a persistent machine	Ephemeral, repo-scoped, never fork-schedulable
One bad shared-workflow release broke every team	Major tag moved without a canary	Canary one repository on the exact patch first
Teams have forked the shared workflow	Upstream changes take too long	Fix turnaround time: policy will not win this
"CI is slow" never gets prioritised	No numbers	Track p95 duration, flake rate, queue time, cost
A six-hour GPU job hit the ceiling	Actions used as compute, not orchestration	Submit to a training platform and gate on the result

The practices that pay off most



Read-only token by default

Set it at organisation level so every new repository is safe without anyone remembering.



OIDC instead of stored keys

Nothing to rotate, nothing to leak. Constrain the trust policy to an environment and verify it is refused elsewhere.



SHA-pin plus Dependabot



Immutable deploy references



Canary the shared workflow

One low-risk repository on the exact patch, green for a day, then move the major tag.



Measure before optimising

Pull run timings from the REST API. The slow step is rarely the one people assume.

The hardening pass every workflow should get

Apply this as a template, or better, as the shared workflow everyone calls.

.github/workflows/ci.yml

Copy

```
permissions:
  contents: read # 1. read-only floor for the workflow

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

jobs:
  build:
    runs-on: ubuntu-latest
    timeout-minutes: 20 # 2. no job may burn six hours
    steps:
      - uses: actions/checkout@v4
        with:
          persist-credentials: false # 3. no push token left in the workspace
      - env:
          TITLE: ${{ github.event.pull_request.title }} # 4. event data as data
        run: ./scripts/check.sh
```



<code>permissions: contents: read</code>	A compromised step's ability to push, tag, or comment
<code>timeout-minutes</code>	Hung jobs, and their cost
<code>persist-credentials: false</code>	A usable token sitting in the workspace for later steps
Event data via <code>env</code>	Shell injection from titles, branch names, comment bodies

Set the organisation default once

In organisation settings, make the default `GITHUB_TOKEN` permission **read repository contents** and require workflows to opt in. That single change makes every new repository safe by default instead of depending on every author remembering.

Pinning that stays fresh

✓ Do

- Pin every third-party action to a full 40-character SHA, version in a comment
- Enable Dependabot for `github-actions` and group the updates
- Restrict the organisation to selected actions plus verified creators
- Fork and vendor anything you depend on critically

✗ Do not

- Reference `@main` or any branch on code you do not own
- Pin and then never update: a frozen action is an unpatched one
- Assume a major tag is immutable; tags can be moved
- Run an unvetted action in a job holding production secrets

`.github/dependabot.yml`

Copy

```
version: 2
updates:
  - package-ecosystem: github-actions
    directory: /
```



actions:

patterns: ['*'] # one pull request a week instead of fifteen

The grouping matters more than it looks. Fifteen separate Dependabot pull requests a week get ignored, and ignored updates are the same as no updates.

Getting rid of stored cloud keys

01 Register GitHub as an identity provider in the cloud account

One-time setup per account, for `token.actions.githubusercontent.com`.

02 Create a role whose trust policy names your repository precisely

Constrain the `sub` claim to a ref, or better an environment: `repo:my-org/my-repo:environment:production`. A wildcard like `repo:my-org/*:*` lets any branch of any repository assume the role.

03 Grant the role only what deploying needs

Not what the team has. This is the step people skip, and the one that bounds the damage.

04 Put `id-token: write` on the job, not the workflow

Only the deploying job needs to mint an identity.

05 Prove the constraint holds

Open a pull request from a throwaway branch with a workflow that tries to assume the role. It must be refused. If it succeeds, the condition is too broad and your approval gate is decorative.

06 Delete the old access keys



YAML

Copy

```
jobs:
  deploy:
    environment: production          # the approval becomes part of the identity
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${vars.DEPLOY_ROLE_ARN}
          aws-region: ${vars.AWS_REGION}
      - run: aws sts get-caller-identity    # confirm identity before acting
```

Verify controls rather than trusting them

BASH

Copy

```
# Which workflows use a dangerous trigger?
grep -rl 'pull_request_target|workflow_run|issue_comment' .github/workflows/

# Which third-party actions are NOT SHA-pinned?
grep -rhoP 'uses:\s*\K(?:actions/)[\w.-]+/[\w.-]+@[\w.-]+' .github/workflows/ \
  | grep -v '@[0-9a-f]{40}$' | sort -u

# Does any run body interpolate event data?
grep -rn 'run:.*${\s*github\.event\. ' .github/workflows/

# Which workflows have no permissions block?
for f in .github/workflows/*.yaml; do
  grep -q '^permissions:' "$f" || echo "no permissions floor: $f"
done
```



security control you have never seen refuse anything is decoration.

The review checklist

Most incidents are prevented at review time, not runtime. A checklist is the only thing that survives a busy week.

Check	Looking for
<code>on:</code> triggers	Any <code>pull_request_target</code> , <code>workflow_run</code> , or <code>issue_comment</code>
Checkout ref	A <code>pull_request_target</code> job checking out <code>head.sha</code>
<code>permissions</code>	Present, minimal, widened per job rather than per workflow
Third-party <code>uses:</code>	SHA-pinned, recognisable source, Dependabot-covered
Interpolation	No <code>\${{ github.event.* }}</code> in a <code>run</code> body or <code>github-script</code>
Secrets	Never echoed, never in a job running untrusted code, environment-scoped where possible
<code>concurrency</code>	Present; cancelling on CI, not cancelling on deploys
<code>timeout-minutes</code>	Present on every job
Cache	Key hashes a lockfile; nothing executable restored and then run
Required checks	Job names unchanged, or branch protection updated in the same change

Make the checklist enforceable



stops depending on whoever picks up the review.

Incident playbooks

A secret leaked into a log

Rotate first. The log is already scraped; assume the credential is compromised regardless of how quickly you deleted it. Then revoke the old credential, delete the log and the run, find the line that printed it, and close the class of problem: secrets through `env`, never interpolated, and OIDC wherever possible so there is nothing to leak.

Deleting the log first and rotating later optimises for the wrong thing.

A third-party action was compromised

Determine which repositories reference it and at what ref. Revoke every secret those workflows could reach, not just the ones you think were used. Replace the reference with a vetted SHA or an internal fork. Then close the class: SHA pinning, an allow-list policy, and Dependabot.

Deploys have been shipping the wrong build

Look at the **cache and artifact names before the code**. A key that never invalidates, or two matrix cells uploading the same artifact name, both produce exactly this and neither appears in a diff. Then make the deployed artifact identifiable by an immutable `sha-` tag or digest so "what is running" has an answer.

Everything went red at once



release, or a runner image change, not your diff.

2m Compare against the last green run

Side by side. Look at the environment, not just the code: runner image version, action versions, cache hit or miss.

5m Re-run the failed job with debug logging

Only the failed job, so you get an answer in two minutes rather than twenty.

8m Bypass the cache

Delete the suspect entry or change the key. A poisoned cache explains failures that make no sense against the diff.

12m Reproduce in the runner image locally

If the command passes in the same image, the difference is credentials, network, or an environment variable.

after Close the loop

Pin what moved, add the assertion that would have caught it, and push the finding into the shared workflow so no other team rediscovers it.

Auditing what earlier levels taught

The habits from Beginner and Mid do not stop mattering. They become things you verify across a fleet rather than remember per workflow. These one-liners are worth running over any repository you inherit.

BASH

Copy

```
# Triggers and filters: which workflows can be started by fork-influenced events?
grep -rln 'pull_request_target\|workflow_run\|issue_comment' .github/workflows/

# Conditions: any job with no guard at all, deploying from any branch?
```



```
grep -rn -B2 'outputs:' .github/workflows/ | grep -i 'token\\|secret\\|password'

# Reuse: which repositories still copy-paste instead of calling the standard?
grep -rL 'uses: my-org/.github/.github/workflows/' .github/workflows/*.yaml

# Guards: any job without a timeout?
for f in .github/workflows/*.yaml; do
  grep -q 'timeout-minutes' "$f" || echo "no timeout: $f"
done
```

Earlier habit	What it becomes at this level
<code>paths-ignore</code> on triggers	A fleet-wide policy, plus stub workflows so required checks still report
<code>if: github.ref = ...</code> on deploys	An environment with branch restrictions, so the gate is not one editable line
Job outputs for strings	An audited boundary: outputs are plain text in run metadata, never credentials
A reusable workflow you wrote	A versioned product with a canary, CODEOWNERS, and a deprecation path
<code>timeout-minutes</code> per job	A default in the shared workflow, so no team has to remember
<code>cache:</code> on a setup action	Layer caching with <code>mode=max</code> , plus awareness that caches are a write surface

Encode, do not document

Every item above is more reliable as a default in the shared reusable workflow than as a line in a wiki. If the compliant path is the one teams already call, compliance stops being a review burden.



The moment other teams depend on your workflow, you are running a platform.

One repository owns the standard, with CODEOWNERS on the workflow folder so changes are reviewed by whoever carries the pager.

Consumers pin a major tag; you move it. Release `v2.7.0` , move `v2` . A fix reaches sixty repositories without sixty pull requests. Breaking changes become `v3` with a migration note, never a silent change inside `v2` .

Canary before moving the tag. One low-risk repository pins the exact patch. Green for a day, then move the major tag. Skip this and one bad release breaks every team at once.

Inputs are a public API. Types, defaults, descriptions. Removing an input is a breaking change even if one repository used it, and the failure lands on somebody else's pull request.

Pass secrets explicitly. `secrets: inherit` hands the callee everything the caller can see. Listing the two it needs documents the contract and bounds the damage from a bad release.

Make the compliant path the easy path. If using the shared workflow is harder than writing a local one, teams fork it and you have sixty variants with your name on them.

The failure mode of shared pipelines

A team needs one small change, cannot get it upstream quickly, and forks. Six months later there are eleven forks and no standard. The fix is **turnaround time**, not a policy. If a reasonable request takes two weeks, forking is the rational choice and you will lose.

Runners and cost

Rule	Why
Hosted by default	Free on public repos, fresh machine, no state to leak



Larger hosted before self-hosted	Same isolation, more cores, no need to operate
Self-hosted only for GPU, licences, or VPC reach	And then ephemeral : one job per runner
Never fork-schedulable	Untrusted code plus persistent state is a compromised machine
Repository-scoped, network-segmented	A compromised runner should not reach production

Windows minutes bill at roughly twice Linux and macOS at roughly ten times. A matrix that includes macOS "for completeness" can become most of your bill, and larger hosted runners are almost always cheaper than the engineering time to run your own fleet.

Performance levers that pay

Lever	Typical gain	Note
Cancel superseded runs	Large on busy repositories	<code>cancel-in-progress</code> on pull requests only
Registry or GHA layer cache with <code>mode=max</code>	Minutes per build	<code>mode=min</code> barely helps a multi-stage build
Shard slow suites across a matrix	Near-linear	Bounded by the slowest shard: shard by duration
Move end-to-end tests to a schedule	Removes them from the critical path	Keep a smoke subset on the PR
Prebuilt job image	Removes install time entirely	<code>container:</code> with an image you build nightly
Larger runners	Sub-linear	Only after measuring



Four numbers turn "CI is slow" from an opinion into a funded priority: **median and p95 duration per workflow, failure rate split into real failures and flakes, queue time, and cost by workflow and runner label.**

YAML

Copy

```
- name: Weekly CI health report
  env:
    GH_TOKEN: ${ secrets.GITHUB_TOKEN }
  run: |
    gh api -X GET '/repos/${ github.repository }/actions/runs' \
      -f per_page=100 --jq '
      [.workflow_runs[] | select(.conclusion != null)
      | { name, conclusion,
        mins: (((.updated_at|fromdate) - (.run_started_at|fromdate)) / 60) }]
      | group_by(.name)[]
      | { workflow: .[0].name, runs: length,
        failed: ([.[] | select(.conclusion = "failure")] | length),
        p95: (sort_by(.mins) | .[(length * 0.95) | floor].mins) }' \
      | tee -a "$GITHUB_STEP_SUMMARY"
```

One caution: debugging actions that open an interactive shell on a runner are useful and are also a live session on a machine holding your secrets. Restrict them to jobs that have none.

Machine-learning pipelines

Validate before you spend. A schema, drift, or row-count check on a cheap runner protects an expensive one. Failing after forty minutes of GPU time on a malformed dataset is an avoidable cost.

Gate on a comparison, not an absolute. "Better than what is deployed" survives a changing dataset; a hard-coded threshold silently becomes either meaningless or a permanent blocker.



Promote from a separate job behind an environment. That buys the approval, the scoped credentials, and a deployment record naming the commit, the dataset version, and the run.

YAML

Copy

```
- name: Gate on a comparison with production
  run: |
    python -m pipeline.compare \
      --candidate "${{ needs.train.outputs.run-id }}" \
      --baseline production \
      --min-delta 0.0
```

The architectural habit underneath all of it: **Actions orchestrates, it does not compute.** The six-hour ceiling is not the real constraint. A CI runner is the wrong place for a long, expensive, retryable workload. Submit it to a platform built for that, wait, and gate on the result.



PREVIOUS

Senior · Interview Review





LEARN

Roadmaps

Student Guides

Courses

The MLOps Practitioner

Sessions

Articles

COMMUNITY

Meet the team

Services for companies

FAQ

Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**



MLOps MENA
COMMUNITY

العربية



Join the Community ↗

